PRACTICAL - 14

Aim: Assemble and test (Task.java + TaskService.java + Driver.java + TaskServiceTest.java)

Code Screenshot:

Visual Studio Code - Task.java

Task.java

```java
public class Task {
    private final String title;
    public Task(String title) { this.title = title; }
    public String getTitle() { return title; }
    public String toString() { return title; }
}
```

Visual Studio Code - TaskService.java

TaskService.java

```java
import java.util.ArrayList;
import java.util.List;

public class TaskService {
    private final List<Task> tasks = new ArrayList<>();

    public void addTask(String title) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("Task cannot be blank");
        }
        tasks.add(new Task(title.trim()));
    }

    public int getCount() { return tasks.size(); }
    public List<Task> getTasks() { return List.copyOf(tasks); }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
import java.util.Scanner;

public class Driver {
    public static void main(String[] args) {
        TaskService service = new TaskService();
        try (Scanner input = new Scanner(System.in)) {
            int choice;
            do {
                System.out.println("1 Add  2 List  3 Count  0 Exit");
                System.out.print("Choice: ");
                choice = input.nextInt();
                input.nextLine();
                switch (choice) {
                    case 1 -> {
                        System.out.print("Task title: ");
                        service.addTask(input.nextLine());
                        System.out.println("Task added.");
                    }
                    case 2 -> service.getTasks().forEach(
                            System.out::println);
                    case 3 -> System.out.println(
                            "Task count: " + service.getCount());
                    case 0 -> System.out.println("Application closed.");
                    default -> throw new IllegalArgumentException(
                            "Invalid action");
                }
            } while (choice != 0);
        }
    }
}
```

Visual Studio Code - TaskServiceTest.java

TaskServiceTest.java

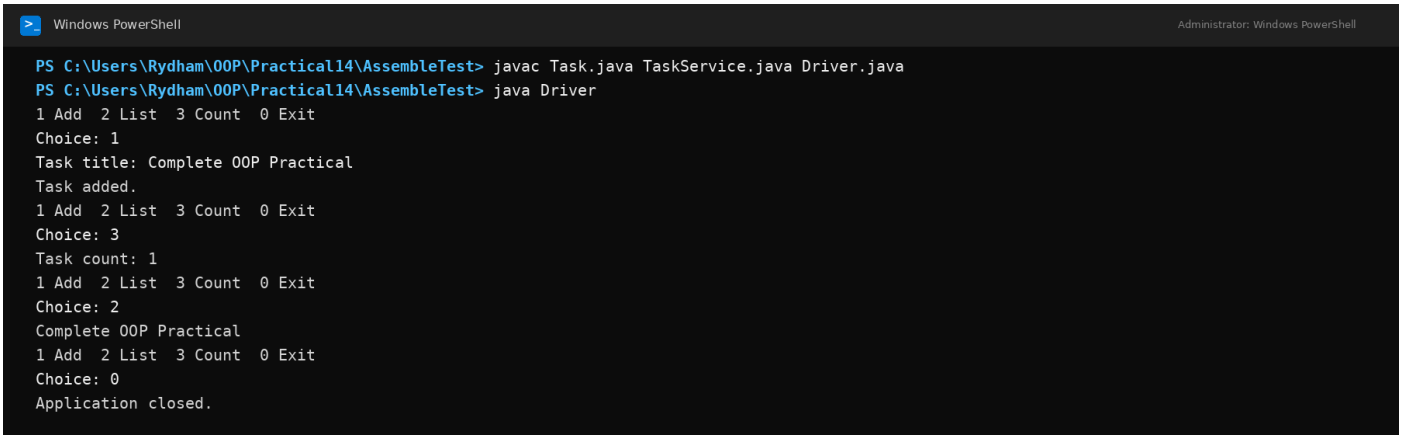
```java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class TaskServiceTest {
    @Test
    void addingTaskIncreasesCount() {
        TaskService service = new TaskService();
        service.addTask("Prepare journal");
        assertEquals(1, service.getCount());
    }

    @Test
    void blankTaskThrowsException() {
        TaskService service = new TaskService();
        assertThrows(IllegalArgumentException.class,
                () -> service.addTask(" "));
    }

    @Test
    void taskTitleIsTrimmed() {
        TaskService service = new TaskService();
        service.addTask("  Java OOP  ");
        assertEquals("Java OOP", service.getTasks().get(0).getTitle());
    }
}
```

Output Screenshot:



Learning Outcome:

- Understood layered software architecture (Service and Driver layers).
- Implemented automated assertion-based unit testing methodologies.
- Verified invariant enforcement and expected exception handling.
- Structured robust service validation rules.
- Gained end-to-end testing competency ahead of capstone project delivery.

PRACTICAL - 14

Aim: Modular packaging (For Advanced Learners)
(Task.java + TaskService.java + TaskStore.java + Driver.java + TaskStoreTest.java)

Code Screenshot:

Visual Studio Code - Task.java

Task.java

src > com > charusat > taskapp > model > Task.java

```java
package com.charusat.taskapp.model;
import java.io.Serializable;
import java.util.Objects;

public class Task implements Serializable {
    private static final long serialVersionUID = 1L;
    private final int id;
    private final String title;

    public Task(int id, String title) { this.id = id; this.title = title; }
    public int getId() { return id; }
    public String getTitle() { return title; }
    public String toString() { return id + " - " + title; }
    public boolean equals(Object obj) {
        if (!(obj instanceof Task other)) return false;
        return id == other.id && Objects.equals(title, other.title);
    }
    public int hashCode() { return Objects.hash(id, title); }
}
```

Visual Studio Code - TaskService.java

TaskService.java

src > com > charusat > taskapp > service > TaskService.java

```java
package com.charusat.taskapp.service;
import com.charusat.taskapp.model.Task;
import java.util.ArrayList;
import java.util.List;

public class TaskService {
    private final List<Task> tasks = new ArrayList<>();
    public void add(Task task) { tasks.add(task); }
    public List<Task> getAll() { return List.copyOf(tasks); }
    public void loadAll(List<Task> saved) {
        tasks.clear();
        tasks.addAll(saved);
    }
}
```

Visual Studio Code - TaskStore.java

TaskStore.java

src > com > charusat > taskapp > storage > TaskStore.java

```java
package com.charusat.taskapp.storage;
import com.charusat.taskapp.model.Task;
import java.io.*;
import java.nio.file.*;
import java.util.List;

public class TaskStore {
    public static void save(List<Task> tasks, Path file) throws IOException {
        try (ObjectOutputStream out = new ObjectOutputStream(
                Files.newOutputStream(file))) {
            out.writeObject(new java.util.ArrayList<>(tasks));
        }
    }

    @SuppressWarnings("unchecked")
    public static List<Task> load(Path file)
            throws IOException, ClassNotFoundException {
        try (ObjectInputStream in = new ObjectInputStream(
                Files.newInputStream(file))) {
            return (List<Task>) in.readObject();
        }
    }
}
```

Visual Studio Code - Driver.java

Driver.java

src > com > charusat > taskapp > app > Driver.java

```java
package com.charusat.taskapp.app;
import com.charusat.taskapp.model.Task;
import com.charusat.taskapp.service.TaskService;
import com.charusat.taskapp.storage.TaskStore;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.Scanner;

public class Driver {
    public static void main(String[] args) throws Exception {
        Path file = Path.of("tasks.dat");
        TaskService service = new TaskService();
        if (Files.exists(file)) {
            service.loadAll(TaskStore.load(file));
            System.out.println("Loaded tasks: " + service.getAll().size());
        }
        try (Scanner input = new Scanner(System.in)) {
            int choice;
            System.out.println("1 Add  2 List  0 Exit");
            do {
                System.out.print("Choice: ");
                choice = input.nextInt();
                input.nextLine();
                switch (choice) {
                    case 1 -> {
                        System.out.print("Task title: ");
                        service.add(new Task(service.getAll().size() + 1,
                                input.nextLine()));
                        System.out.println("Task added.");
                    }
                    case 2 -> service.getAll().forEach(System.out::println);
                    case 0 -> System.out.println("Closing application.");
                    default -> throw new IllegalArgumentException("Invalid action");
                }
            } while (choice != 0);
        }
        TaskStore.save(service.getAll(), file);
        List<Task> restored = TaskStore.load(file);
        System.out.println("Saved tasks: " + service.getAll().size());
        System.out.println("Reload verified: " + restored.equals(service.getAll()));
    }
}
```

Visual Studio Code - TaskStoreTest.java

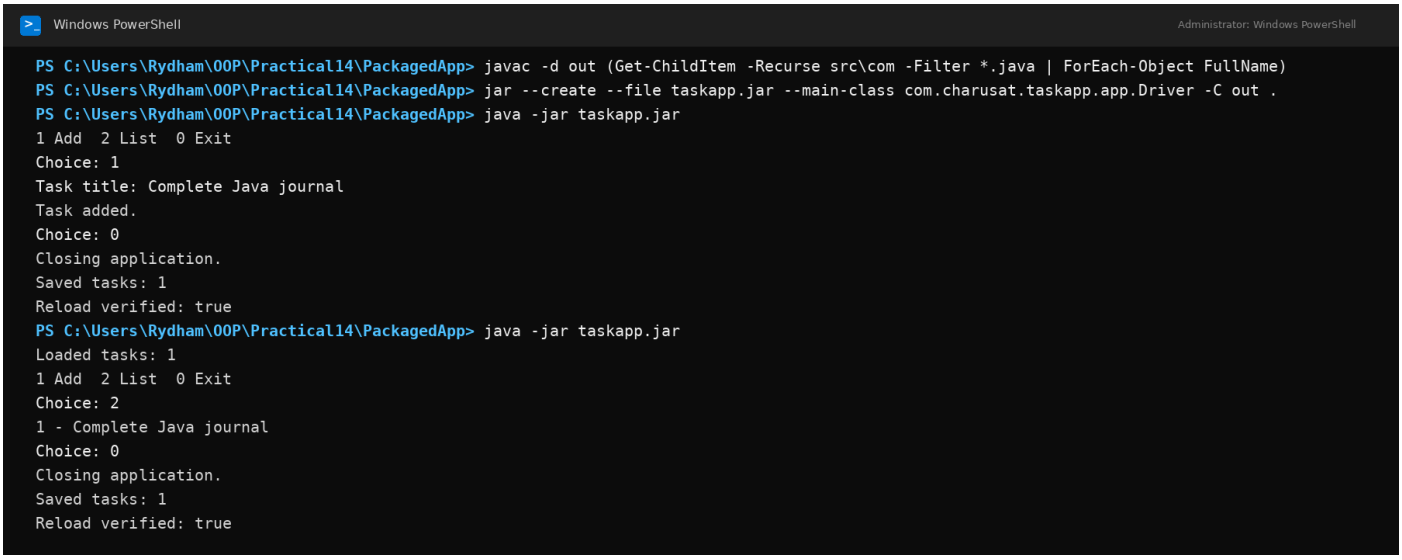
TaskStoreTest.java

src > test > java > com > charusat > taskapp > storage > TaskStoreTest.java

```java
package com.charusat.taskapp.storage;
import com.charusat.taskapp.model.Task;
import org.junit.jupiter.api.Test;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import static org.junit.jupiter.api.Assertions.assertEquals;

class TaskStoreTest {
    @Test
    void savedTasksReloadUnchanged() throws Exception {
        Path file = Files.createTempFile("tasks", ".dat");
        List<Task> original = List.of(
                new Task(1, "Study Java"), new Task(2, "Build JAR"));
        TaskStore.save(original, file);
        List<Task> loaded = TaskStore.load(file);
        assertEquals(original, loaded);
        Files.deleteIfExists(file);
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered Java package encapsulation and organizational structures.
- Learned how classes are bundled and deployed as runnable JAR archives.
- Verified modular integrity across package boundaries.
- Ensured application lifecycle continuity across deployment artifacts.
- Achieved industry-ready software packaging and delivery readiness.